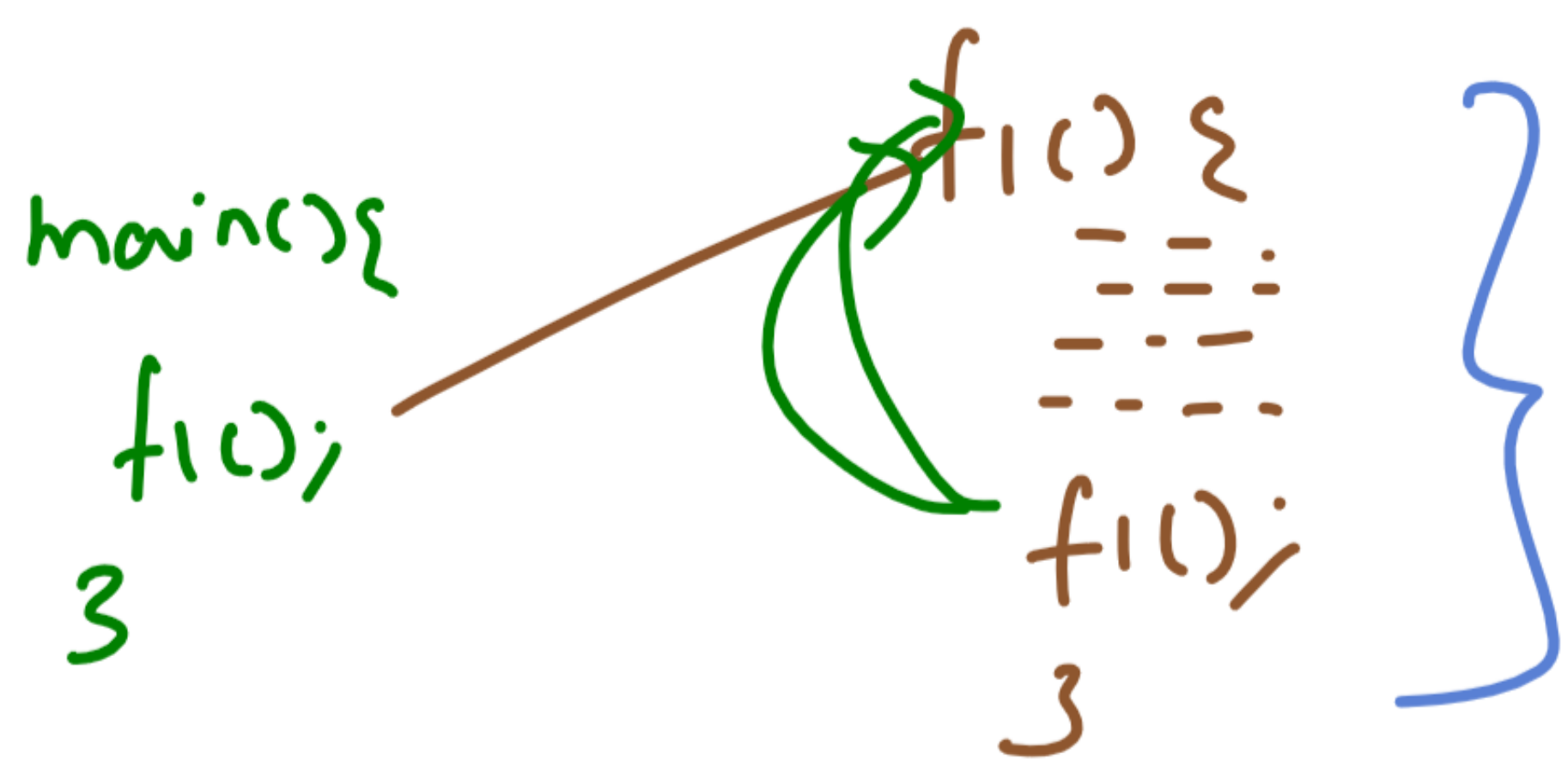


Recursion - Function calls itself with appropriate base condition



→ base condition is used to stop recursion call

→ return means swipe out from the call stack.

1) Print n to 1 without loop.

n = 5

OP: 5 4 3 2 1

Expectation

Pd(5) ⇒ 5 4 3 2 1

Pd(4) ⇒ 4 3 2 1

Reality

n = 5
5 4 3 2 1
Pd(4)

```
static void printDec(int n){
    if(n==0) return; — ①
    System.out.print(n+" "); — ②
    printDec(n-1); — ③
}
```

Pd(0)	0	①
Pd(1)	1	①, ②, ③
Pd(2)	2	①, ②, ③
Pd(3)	3	①, ②, ③
Pd(4)	4	①, ②, ③
Pd(5)	5	①, ②, ③
	n	statements

Pd(5) — Pd(4) — Pd(3)
5 4 3

Pd(2) — Pd(1) — Pd(0)
2 1

5 4 3 2 1

Print Increasing

n = 5

OP: 1 2 3 4 5

PI(5,1) — PI(5,2) — PI(5,3) — PI(5,4) — PI(5,5) — PI(5,6)
1 2 3 4 5


```

if(n==0) return; —①
printInc(n-1); —②
System.out.print(n+" "); —③

```

1 2 3 4 5

Expectation

PI(5) = 1 2 3 4 5

PI(4) = 1 2 3 4

Reality

n=5

1 2 3 4 5

PI(4)

PI(0)	0	①
PI(1)	1	①, ②, ③
PI(2)	2	①, ②, ③
PI(3)	3	①, ②, ③
PI(4)	4	①, ②, ③
PI(5)	5	①, ②, ③

n statements.

Expectation.

Factorial

f(5) = 120
f(4) = 24

Reality

n=5

5 * f(4)

n * f(n-1)

```

int factorial(int n) {
    // code here
    if(n==0) return 1;
    return n*factorial(n-1);
}

```

f(5) ← 120
5 * f(4) ← 24
4 * f(3) ← 6
3 * f(2) ← 2
2 * f(1) ← 1
1 * f(0) ← 1

n=1234 op=10

Sum of Digits

sod(1234) = 10

sod(123) = 6

Reality

n=1234

sod(123) + 4 ⇒ sod(n/10) + n%10

```

if(n<10) return n;
return sumOfDigits(n/10)+n%10;

```


Compute x^n

$$\left. \begin{array}{l} x=2 \\ n=5 \end{array} \right\} \text{OP} = 32$$

$$\text{Pow}(2,5) = 32$$

$$\text{Pow}(2,4) = 16$$

$$\text{Pow}(2,5) \{$$

$$2 * \text{Pow}(2,4)$$

$$x * \text{Pow}(x, n-1)$$

$$\}$$

$$2^5 \Rightarrow 2^1 * 2^4 \Rightarrow 2^5$$

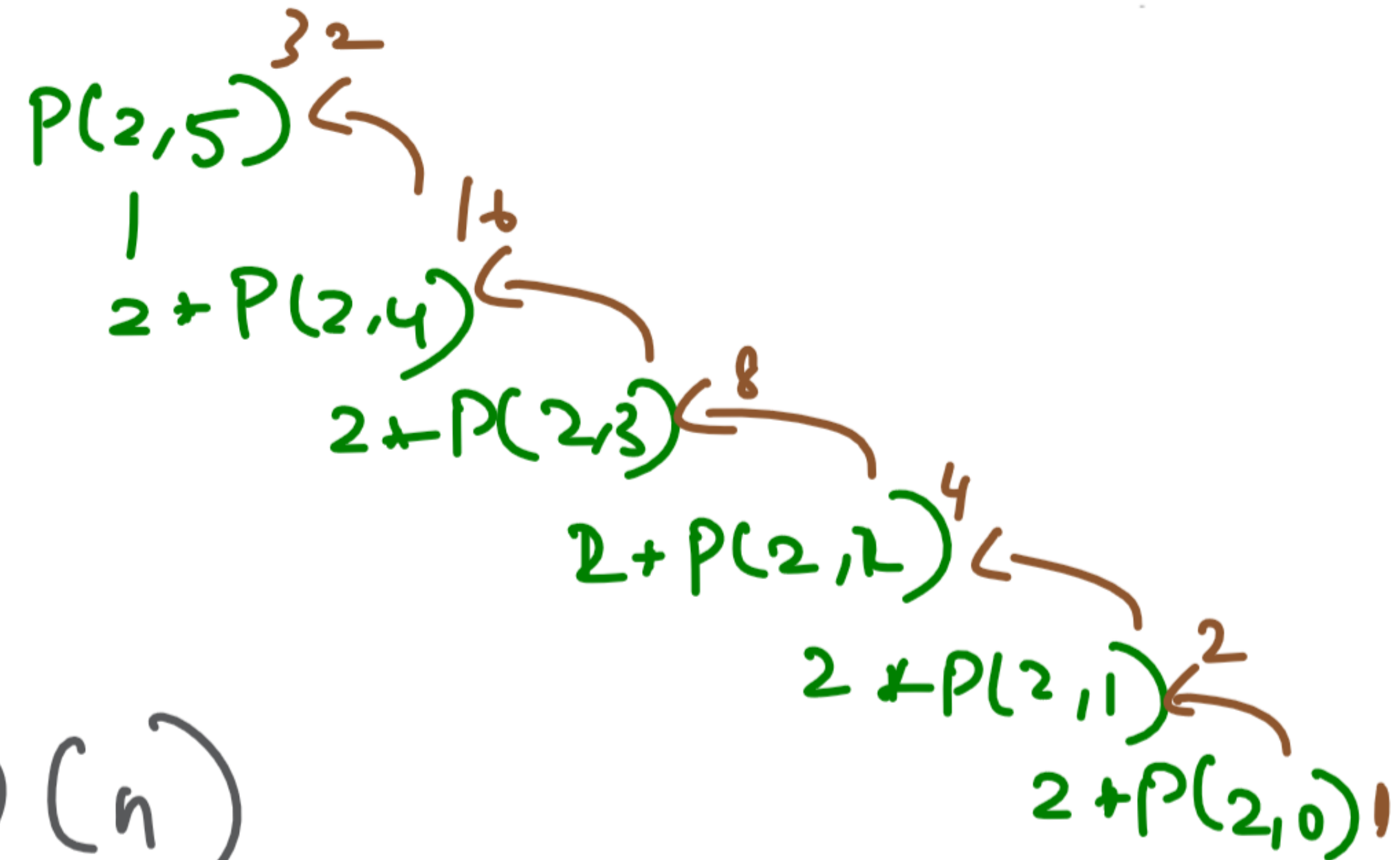
$$3^4 \Rightarrow 3^1 * 3^3 \Rightarrow 3^4$$

$$\Rightarrow \text{T.C} = O(n)$$

$$1 \text{ iteration} = \frac{1}{10^8} \text{ sec}$$

$$10^{10} * \frac{1}{10^8} = 10^2 \text{ sec} \approx 317 \text{ years}$$

```
static int pow(int x, int n){
    if(n==0) return 1;
    return x*pow(x, n-1);
}
```



$$2^{10} = 2^5 * 2^5$$

$$2^8 = 2^4 * 2^4$$

$$2^6 = 2^3 * 2^3$$

$$2^4 = 2^2 * 2^2$$

$$2^9 = 2^4 * 2^4 * 2$$

$$2^7 = 2^3 * 2^3 * 2$$

$$2^5 = 2^2 * 2^2 * 2$$

$$2^3 = 2^1 * 2^1 * 2$$

$$x^n = x^{n/2} * x^{n/2}$$

$$x^n = x^{n/2} * x^{n/2} * x$$

$$\left. \begin{array}{l} 2^{100} \Rightarrow 2^{50} \Rightarrow 2^{25} \Rightarrow 2^{12} \Rightarrow 2^6 \\ 2^3 \Rightarrow 2^1 \Rightarrow 2^0 \end{array} \right\} (\log_2 n)$$

$$\text{Pow}(2,6) = 64$$

$$\text{Pow}(2,3) = 8$$

```
static int pow(int x, int n){
    if(n==0) return 1;
    int xpn2=pow(x, n/2);
    if(n%2==1){
        return xpn2*xpn2*x;
    }else{
        return xpn2*xpn2;
    }
}
```

Fibonacci Series

0 1 1 2 3 5 8 13 ...

0 1 2 3 4 5 6 7

$$f(7) = 13$$

$$f(6) = 8$$

$$f(5-2) = 5$$

```
class Solution {
    public int nthFibonacci(int n) {
        // code here
        if(n==0 || n==1) return n;
        return nthFibonacci(n-1)+nthFibonacci(n-2);
    }
}
```

$$f(n) = f(n-1) + f(n-2)$$

Subsequence $\rightarrow$ Part of the string but no need to be continuous, order is matter.

"abc" — substring
a, ab, abc
b, bc, c, ac α

"abc" — a, ab, abc, b, bc, c, ac, ca α

$\rightarrow$ all the substrings are subsequence.

$\rightarrow$ all the subsequence are not a substring.

IP:

str = "abcd" $\Rightarrow 4 = 16$

op = ["", a, ab, ac, ad, abc, abd, acd, abcd, b, bc, bd, bcd, c, cd, d]

str = abc $\Rightarrow 3 = 8$

op = ["", a, ab, ac, abc, b, bc, c]

Total no of sub-sequence $\} = 2^{\text{len(str)}}$

Ex Recursion

gss(abc) $\Rightarrow$ ["", a, ab, ac, abc, b, bc, c]

gss(bc) $\Rightarrow$ ["", b, bc, c]

recursion

gss(str)

ch = a

ans[] = gss(str) $\Rightarrow$ ["", b, bc, c]

$gss(abcd) = ["", \check{a}, \check{a}b, \check{a}c, \check{a}d, \check{a}bc, \check{a}bd, \check{a}cd, \check{a}bcd, b, bc, bd, bcd, c, cd, d]$

$gss(bcd) = ["", b, bc, bcd, bd, c, cd, d]$

$str = "abcd"$

$ch = a$

$ans[] = gss(bcd) = ["", b, bc, bcd, bd, c, cd, d]$

$sol = [a, ab, abc, abcd, abd, ac, acd, ad, "", b, bc, bcd, bd, c, cd, d]$

$gss(\overset{str}{abc}) \{$

$ch = a \text{ — } str.charAt(0)$

$ans[] = gss(bc) = ["", b, c, bc]$

$sol = [a, "", ab, b, ac, c, abc, bc]$

$str = abc$

$ch = a$

$ans[] = gss(bc) \leftarrow [b, "", bc, c]$

$ch = b$

$ans[] = gss(c) \leftarrow ["", c]$

$sol[] = [b, "", bc, c]$

$ch = c \text{ — } ["", c]$

$ans[] = gss("") \text{ — } [""]$

$sol[] = \{c, ""\}$


```

public static ArrayList<String> getss(String str){//abc
//write your code here
    if(str.length()==0){
        ArrayList<String> base=new ArrayList<>();
        base.add("");
        return base;
    }
    char ch=str.charAt(0);//a
    ArrayList<String> ans=getss(str.substring(1));//["","b","c","bc"]
    ArrayList<String> sol=new ArrayList<>();
    for(String subans:ans){
        sol.add(ch+subans);
        sol.add(subans);
    }
    return sol;
}

```

["",c,d,cd]

gss(cd) [cd, d, c, ""]

ch=c

ans[] = gss(d) [d, ""].

ch=d

ans[] = gss("") = [""]

base = [""]

sol = [d, ""].

sol = [cd, d, c, ""]

